Welcome to C++ Programming (Week 3)

Let's Learn How to Talk to Computers

C++ and Coding: What You Need to Know

What is coding?
Coding is giving instructions to a computer so it knows what to do. You write those instructions using a programming language such as C++.

What can we do with coding?
Games, calculators, websites, and even parts of operating systems!

What will we do today?
Learn basic C++ commands and write some fun code together!

C++ Compiling Process

- **You Write the Code**
  You type your C++ program in Xcode (e.g. main.cpp).

- **Xcode Compiles It**
  When you click **Run**, Xcode:

  - Uses the **C++ compiler** (e.g. clang++) behind the scenes.
  - **Checks for syntax errors**.
  - If there are errors, it stops and shows them in the **Issue Navigator**.

- **If No Errors, It Builds an Executable**
  The compiler converts your .cpp code into a machine-code file that your Mac can run.

- **Xcode Runs the Program**
  The compiled program executes, and your program starts running.

Course Overview

Lesson 1 - Foundations of C++ — Output, Variables, Conditional statements, and Loops

Lesson 2 - Arrays, and String Functions

Lesson 3 - Functions and Pointers

Lesson 4 - Structs and Basic Classes

Lesson 5 - Advanced OOP: Inheritance

Lesson 6 - Maps and Vectors

Lesson 7 – Common Data Structures

Lesson 8 – Sorting Algorithms

How To Use OneCompiler?

Step 1 – Go to the Website

1. Open your web browser.
2. Type in: **https://onecompiler.com/cpp**
3. You'll see a text editor on the left, an output panel on the right, and a "Run" button at the top.

Step 2 – Understand the Layout

- **Left side:** where you write your C++ code.
- **Top bar:** has the **Run**, **Save**, and **Share** buttons.
- **Right side (Output):** shows your program's output or any error messages.

Your First Program in C++

```cpp
#include <iostream>
using namespace std;

0 references
int main()
{
    cout << "Hello, world!" << endl;
    return 0;
}
```

```
Hello, World!
```

cout << "Hello, world!" << endl;

- cout means **console output** → it's what we use to **print text** to the screen.
- << is called the **insertion operator**. It is akin to saying: "send this to the screen."
- "Hello, world!" is the **message** we're printing.
- << endl ends the current line and moves subsequent output to the next line. You can achieve the same goal using "\n" which is a **newline character placed inside the string**. You put \n **exactly where you want the new line to start** within the text.

So this line tells the computer:
"Print *Hello, world!* to the screen, then go to a new line."

Variables and Inputs

```cpp
//Types of variables in C++
int age = 14; // int - stores integers
double gpa = 3.75; // double - stores decimal numbers
bool isStudent = true; // bool - stores true or false values
string name = "Alex"; // string - stores texts
char grade = 'A'; // char - stores a single character

//User input example
int age;
cout << "Enter your age: ";
cin >> age;
```

cout displays a message.

cin lets the user type in a value which is stored in a variable.

```
Enter your age: 16
```

A **variable** stores a piece of information (such as a number or a word) that your program can use later. Each variable has a **type** that tells the computer what kind of data it holds.

- **int** – stores integers (e.g. int age = 14;)
- **double** – stores decimal numbers (e.g. double gpa = 3.75;)
- **bool** – stores true or false values (e.g. bool isStudent = true;)
- **string** – stores texts (e.g. string name = "Alex";)
- **character –** stores a single character (e.g. char grade = 'A';)

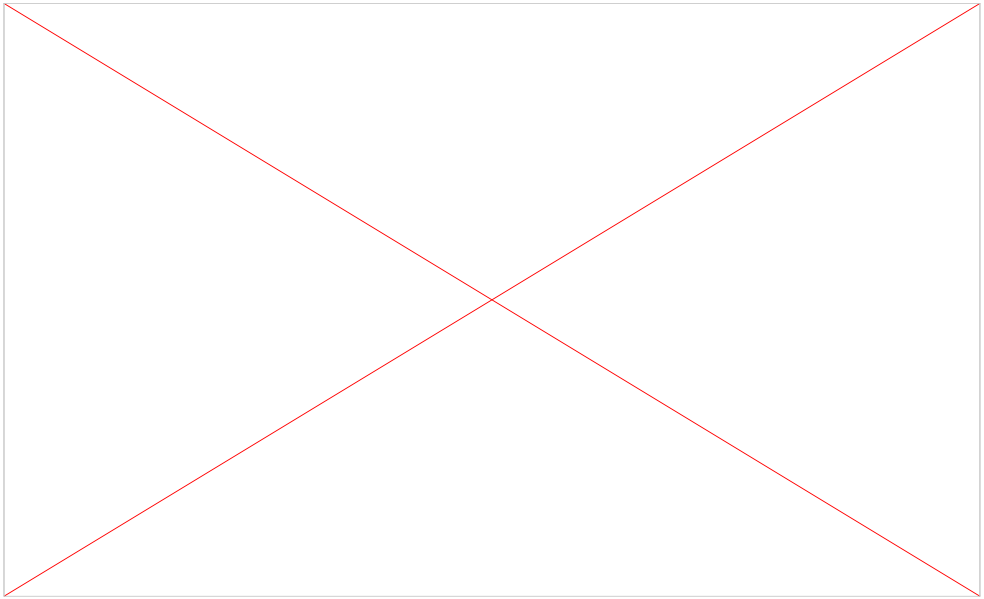
You declare a variable in the following way:

type name = value;

If Statements

```
int age = 16;
if (age >= 13) {
    cout << "You are a teen!" << endl;
}
else {
    cout << "You are a kid!" << endl;
}
```

```
You are a teen
```



For and While Loops

```
//For loop
cout << "For loop: " << endl;
for (int i = 0; i <= 5; i++) {
    cout << i << " ";
}

cout << endl << endl;

//While loop
cout << "While loop: " << endl;
int x = 0;
while (x <= 5) {
    cout << x << " ";
    x++;
}
```

```
For loop:
0 1 2 3 4 5

While loop:
0 1 2 3 4 5
```

For loop: use when you know how many times to repeat.

1. Set a counter value (e.g. i = 0).
2. Check a condition (e.g. i <= 5).
3. If true, run the code.
4. Update the counter (e.g. i++).
5. Repeat until the condition is false.

While loop: use when you don't know exactly how many times to repeat.

1. Start with a condition.
2. If the condition is true, run the code.
3. Update variable(s) inside the loop unless you are intentionally creating an infinite loop which is a loop that never ends.
4. Go back and check the condition again.
5. Repeat steps 2-4 until the condition is false.

What is an Array?

An **array** stores multiple values of the same type in one variable.

- int – data type of the array (stores integers)

elements.

```cpp
int numbers[5] = {1, 2, 3, 4, 5};
```

Declaring and Initializing Arrays

```cpp
int nums[5];
for (int i = 0; i < 5; i++) {
    nums[i] = i * 10;
}
```

Using a Loop to Initialize

- `int nums[5];` → creates an empty array of 5 integers.
- `for (int i = 0; i < 5; i++)` → repeats 5 times for each index.
- `nums[i] = i * 10;` → assigns a value based on the index:
- `nums[0] = 0`
- `nums[1] = 10`
- `nums[2] = 20`, etc.

This method is useful for **patterns** or **large arrays** where typing all values manually is hard.

Accessing Array Elements

```cpp
cout << scores[0];
scores[2] = 100;
```

- `scores[0]` → accesses the **first element** of the array.
- `cout << scores[0];` → prints that value.
- `scores[2] = 100;` → changes the **third element** to 100.
- ⚠️ Indices start from **0**, so valid indices go from 0 to (size–1).
- Accessing an invalid index (like `scores[10]`) causes errors.

Looping Through Arrays

```cpp
for (int i = 0; i < 5; i++) {
    cout << scores[i] << " ";
}
```

- `for` loop → repeats code for each index in the array.
- `int i = 0;` → start counting at the first element.
- `i < 5;` → stop when reaching the last index (4).
- `scores[i]` → accesses each element one by one.
- `cout << ...` → prints all elements separated by spaces.

Arrays with Different Types

```cpp
string fruits[4] = {"Apple", "Banana", "Cherry", "Mango"};
```

```cpp
char grade[5] = {'A', 'B', 'C', 'D', 'F'};
```

```cpp
double prices[4] = {2.99, 5.49, 1.25, 4.75};
```

Double Array

- `double` → stores **decimal (floating-point)** numbers.
- Commonly used for prices, averages, or measurements.
- Example: `prices[2] = 1.25`.

Character Array

- `char` → stores **single characters** (letters, digits, symbols).
- Character values are placed in **single quotes** `' '`.
- Example: `grade[0]` → `'A'`.

String Array

- `string` → each element stores **text**, not numbers.
- `fruits` → name of the array.
- `[4]` → there are 4 string elements.
- The array holds multiple **words or names**.

Strings

```cpp
string name = "Alice";
```

- `string` → data type from the `<string>` library.
- `name` → variable holding text.
- `"Alice"` → the actual text stored.
- Strings can hold letters, numbers, and spaces.

Strings

```cpp
string word = "hello";
cout << word.length();  // Output: 5
```

```cpp
string word = "hello";
cout << word.substr(1, 3);  // Output: "ell"
```

`.length()` → returns the **number of characters** in the string.

- Example: `word.length()` → 5
- Counts spaces and punctuation too (e.g., `"hi there"` → 8).

`.substr(start, length)` → extracts a **substring** starting at `start` for `length` characters.

- Example: `word.substr(1, 3)` → `"ell"`
- If `length` is too long, it just goes to the end of the string.

Strings

```cpp
string word = "hello";
cout << word.find("l");  // Output: 2
```

`.find("text")` → finds the **index where the given text first appears**.

- Example: `word.find("l")` → 2
- Returns `string::npos` if not found (useful for checking if a letter/word exists).

```cpp
string word = "hello";
word.append("!");
cout << word;  // Output: "hello!"
```

`.append("text")` → **adds text to the end** of a string.

Introduction to Functions in c++

A **function** is a small piece of code that does **one specific job**.
Examples of jobs a function can do:

- Add two numbers
- Print a message
- Calculate something

Functions help us reuse code instead of rewriting it.

Why Use Functions?

Functions are helpful because they:

- **Prevent repeating code**
- **Make programs easier to read**
- **Organize** your program
- **Break big tasks** into smaller ones

Parts of a C++ Function?

Return type — what value it gives back (ex: int)

Name — what we call it

Parameters — the input values

Body — the code inside the function

Function Structure

```scss
returnType functionName(parameters) {
    // code
}
```

Explanation:

- `returnType` → type of answer the function gives back
- `functionName` → what you call to use it
- `parameters` → inputs (can be empty)
- `{ }` → code the function runs

Example: Add Two Numbers

```cpp
int add(int x, int y) {
    return x + y;
}
```

Explanation:

- `int` → the function returns an integer
- `add` → function name
- `x` and `y` → inputs (parameters)
- `return x + y` → sends the sum back to whoever called it

Calling a Function

```sql
int result = add(3, 5);
cout << result;
```

Explanation:

- `add(3, 5)` runs the function
- It returns **8**
- `result` stores that 8
- Then we print it

Functions with No Parameters

```csharp
void sayHello() {
    cout << "Hello!";
}
```

Explanation:

- `void` → returns nothing
- No parameters → it doesn't need any input
- Prints "Hello!" every time it is called

Function With No Return Value

```cpp
void printNumber(int n) {
    cout << n;
}
```

Explanation:

- Still has a parameter (n)
- But `void` means it does not return anything
- It simply **prints** the number

Full C++ Program with Functions

```cpp
#include <iostream>
using namespace std;

int add(int x, int y) {
    return x + y;
}

int main() {
    int result = add(4, 6);
    cout << "The answer is: " << result;
}
```

Explanation:

- `add` function is defined above main
- `main()` calls `add(4, 6)`
- The answer (10) is printed

Mini-Challenge Time

Try solving these in C++:

1. Print your pet's information.
2. Ask for a number and print it 5 times.
3. Print from 1 to 10.
4. Ask the user to input a number and check if it's even or odd, and if it's positive, negative, or zero.
5. Guess-the-number game (while loop).

Mini-Challenge Time (continued)

6. Multiplication Table

Ask for a number and print its multiplication table from 1 to 10 using a loop.

7. Simple Calculator

Ask the user for two numbers and an operation (+, -, *, or /). Print the result.

8. Factorial Calculator

Ask the user for a number and compute its factorial using a for or while loop.
(Example: 5! = 5 × 4 × 3 × 2 × 1 = 120)

Mini-Challenge Time (continued)

9. Number Stats Analyzer (Hard)

Ask the user how many numbers they want to enter. Then:

- Use a loop to input all numbers.
- Keep track of:
  - The largest and smallest number
  - How many are even
  - The sum of all numbers
- Print the summary at the end.

Mini-Challenge (continued)

10. Detect Palindromic Numbers

Ask the user for a number. Reverse it using math (no strings) and check if it's a palindrome (same forward and backward).

11. String Palindrome Checker

Ask the user to enter a string (a word).

Check if the string is a palindrome (reads the same forward and backward).

12. Given an array of integers, find the sum of all elements. Input: arr = {1, 2, 3, 4, 5}

13. Find the largest element in the array. Input: arr = {10, 25, 3, 47, 18}

Mini-Challenge (continued)

14. Reverse the order of elements in an array. Input: arr = {1, 2, 3, 4, 5}

15. Count how many times each number appears in an array. Input: arr = {1, 2, 2, 3, 1, 4, 2}

16. Use `.length()` to find how many characters are in a string.

17. Use `.substr(start, length)` to extract part of a string.

18. Use `.find()` to locate the first position of a given character or substring.

19. Use `.append()` and + operator to join two strings.

Mini-Challenge (continued)

20. Write a function that returns the square of a number.
21. Write a void function that prints your name.
22. Write a function that takes an array and returns the sum of its elements.
23. Count Vowels in a String
24. Compare Two Strings
25. Last Character of a String

Challenge 1 solution

```
cout << "Pet Name: Mochi\n";
cout << "Pet Type: Cat\n";
cout << "Pet Age: 2\n\n";
```

```
Pet Name: Mochi
Pet Type: Cat
Pet Age: 2
```

Challenge 2 Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int number;

    cout << "\nEnter a number: ";
    cin >> number;

    for (int i = 0; i < 5; i++) {
        cout << number << endl;
    }
}
```

```
Enter a number: 5
5 5 5 5 5
```

Challenge 3 Solution

```cpp
#include <iostream>
using namespace std;

int main() {

    cout << "\nCounting from 1 to 10:" << endl;
    for (int i = 1; i <= 10; i++) {
        cout << i << " ";
    }
    cout << endl;
}
```

```
Numbers from 1 to 10:
1 2 3 4 5 6 7 8 9 10
```

Challenge 4 Solution

```cpp
#include <iostream>
using namespace std;

int main() {

    int userNum;
    cout << "\nEnter a number to check: ";
    cin >> userNum;

    // Even or Odd
    if (userNum % 2 == 0)
        cout << "It is even." << endl;
    else
        cout << "It is odd." << endl;

    // Positive, Negative, or Zero
    if (userNum > 0)
        cout << "It is positive." << endl;
    else if (userNum < 0)
        cout << "It is negative." << endl;
    else
        cout << "It is zero." << endl;

}
```

```
Enter a number to check: 4
The number is even.
The number is positive.
```

Challenge 5 Solution

```cpp
#include <iostream>
using namespace std;

int main() {

    int secret = 7;
    int guess;

    cout << "Guess the number (between 1 and 10): ";
    cin >> guess;

    while (guess != secret) {
        if (guess < secret) {
            cout << "Too small! Try again: ";
        }
        else {
            cout << "Too large! Try again: ";
        }
        cin >> guess;
    }

    cout << "You guessed it!" << endl;

    return 0;
}
```

```
Guess the secret number between 1 and 10: 5
Too small. Try again: 8
Too large. Try again: 7
Correct! You guessed the secret number.
```

Challenge 6 Solution

```cpp
#include <iostream>
using namespace std;

int main() {

    int num;
    cout << "Enter a number: ";
    cin >> num;

    for (int i = 1; i <= 10; i++) {
        cout << num << " x " << i << " = " << num * i << endl;
    }

    return 0;
}
```

```
Multiplication table for 4:
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40
```

Challenge 7 Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    double num1, num2;
    char op;

    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter an operator (+, -, *, /): ";
    cin >> op;
    cout << "Enter second number: ";
    cin >> num2;

    if (op == '+') {
        cout << "Result: " << num1 + num2 << endl;
    }
    else if (op == '-')
        cout << "Result: " << num1 - num2 << endl;
    else if (op == '*')
        cout << "Result: " << num1 * num2 << endl;
    else if (op == '/')
        if (num2 != 0)
            cout << "Result: " << num1 / num2 << endl;
        else
            cout << "Cannot divide by zero!" << endl;
    else
        cout << "Invalid operator." << endl;

    return 0;
}
```

```
Enter first number: 6
Enter an operator (+, -, *, /): +
Enter second number: 5
Result: 11
```

Challenge 8 Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int num;
    unsigned long long factorial = 1;

    cout << "Enter a number: ";
    cin >> num;

    for (int i = num; i >= 1; i--) {
        factorial *= i;
    }

    cout << num << "! = " << factorial << endl;

    return 0;

}
```

```
Enter a positive number: 4
Factorial of 4 is 24
```

Challenge 9 Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int count;
    cout << "How many numbers do you want to enter? ";
    cin >> count;

    // Declare variables
    int number;
    int largest, smallest;
    int evenCount = 0;
    int sum = 0;

    for (int i = 0; i < count; i++) {
        cout << "Enter number " << (i + 1) << ": ";
        cin >> number;

        // Add to sum for average
        sum += number;

        // Count even numbers
        if (number % 2 == 0) {
            evenCount++;
        }

        // Set largest/smallest on first loop
        if (i == 0) {
            largest = smallest = number;
        }
        else {
            if (number > largest) {
                largest = number;
            }
            if (number < smallest) {
                smallest = number;
            }
        }
    }

    // Print the results
    cout << "\nSummary:\n";
    cout << "Largest number: " << largest << endl;
    cout << "Smallest number: " << smallest << endl;
    cout << "Even numbers: " << evenCount << endl;
    cout << "Sum: " << sum << endl;

    return 0;
}
```

```
How many numbers do you want to enter? 4
Enter number 1: 1
Enter number 2: 2
Enter number 3: 3
Enter number 4: 4

Summary:
Largest number: 4
Smallest number: 1
Even numbers: 2
Sum: 10
```

Challenge 10 Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    int original = num;
    int reversed = 0;

    while (num > 0) {
        int digit = num % 10;
        reversed = reversed * 10 + digit;
        num /= 10;
    }

    if (reversed == original) {
        cout << "The number is a palindrome." << endl;
    }
    else {
        cout << "The number is not a palindrome." << endl;
    }

    return 0;
}
```

```
Enter a number: 120
120 is not a palindrome.
```

Challenge 11 Solution

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string word;
    cout << "Enter a word: ";
    cin >> word;

    int isPalindrome = 1; // assume it is

    int left = 0;
    int right = word.length() - 1;

    while (left < right) {
        if (word[left] != word[right]) {
            isPalindrome = 0;
            break;
        }
        left++;
        right--;
    }

    if (isPalindrome) {
        cout << "The word is a palindrome." << endl;
    }
    else {
        cout << "The word is not a palindrome." << endl;
    }

    return 0;
}
```

```
Enter a word: woow
woow is a palindrome.
```

Challenge 12 Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int sum = 0;

    for (int i = 0; i < 5; i++) {
        sum += arr[i];
    }

    cout << "Sum = " << sum;
    return 0;
}
```

```ini
Sum = 15
```

Challenge 13 Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {10, 25, 3, 47, 18};
    int maxVal = arr[0];

    for (int i = 1; i < 5; i++) {
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }

    cout << "Maximum = " << maxVal;
    return 0;
}
```

```ini
Maximum = 47
```

Challenge 14 Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = 5;

    cout << "Original array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";

    // Reverse the array
    for (int i = 0; i < n / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }

    cout << "\nReversed array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```

```php
Original array: 1 2 3 4 5
Reversed array: 5 4 3 2 1
```

Challenge 15 Solution

cpp

```cpp
#include <iostream>
#include <map>
using namespace std;

int main() {
    int arr[] = {1, 2, 2, 3, 1, 4, 2};
    int n = 7;
    map<int, int> count;

    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    cout << "Frequency of each number:\n";
    for (auto pair : count) {
        cout << pair.first << " appears " << pair.second << " times\n";
    }

    return 0;
}
```

perl

```
Frequency of each number:
1 appears 2 times
2 appears 3 times
3 appears 1 times
4 appears 1 times
```

Challenge 16 Solution

cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "Hello World";
    cout << "The string is: " << text << endl;
    cout << "Number of characters: " << text.length();
    return 0;
}
```

csharp

```
The string is: Hello World
Number of characters: 11
```

Challenge 17 Solution

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "Programming";
    string part = text.substr(3, 5);
    cout << "Extracted substring: " << part;
    return 0;
}
```

```yaml
Extracted substring: gram
```

Challenge 18 Solution

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "Programming";
    string part = text.substr(3, 5);
    cout << "Extracted substring: " << part;
    return 0;
}
```

```yaml
Extracted substring: gram
```

Challenge 20 Solution

```cpp
#include <iostream>
using namespace std;

int square(int x) {
    return x * x;
}

int main() {
    int result = square(7);
    cout << "Square is: " << result;
    return 0;
}
```

```csharp
Square is: 49
```

Challenge 21 Solution

```cpp
#include <iostream>
using namespace std;

void printName() {
    cout << "Aria Thunderstone" << endl;
}

int main() {
    printName();
    printName();
    printName();
}
```

```nginx
Aria Thunderstone
Aria Thunderstone
Aria Thunderstone
```

Challenge 22 Solution

```cpp
#include <iostream>
using namespace std;

int sumArray(int arr[], int size) {
    int sum = 0;
    for(int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}

int main() {
    int nums[5] = {2, 4, 6, 8, 10};
    int result = sumArray(nums, 5);
    cout << "Sum: " << result;
}
```

Printed Output

```makefile
Sum: 30
```

Challenge 23 Solution

cpp

```cpp
#include <iostream>
using namespace std;

int countVowels(string s) {
    int count = 0;
    for(int i = 0; i < s.length(); i++) {
        char c = tolower(s[i]);
        if(c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
            count++;
    }
    return count;
}

int main() {
    string text = "Hello World";
    int result = countVowels(text);
    cout << "Vowel count: " << result;
}
```

Printed Output

yaml

```yaml
Vowel count: 3
```

Challenge 24 Solution

```cpp
#include <iostream>
using namespace std;

int compareStrings(string a, string b) {
    if (a == b) return 0;
    if (a > b) return 1;
    return -1;
}

int main() {
    string s1 = "apple";
    string s2 = "banana";

    int result = compareStrings(s1, s2);
    cout << "Comparison result: " << result;
}
```

rinted Output

```sql
Comparison result: -1
```

Brain Teaser

You have two ropes, and each takes exactly one hour to burn. How would you use them to time exactly 15 minutes? Note that the ropes are of uneven densities, so half the rope length-wise does not necessarily take half an hour to burn.

Brain Teaser Solution

1. Light rope 1 at both ends and rope 2 at one end.
2. When two flames on rope 1 meet, 30 minutes will have passed. Rope 2 has 30 minutes left of burn-time.
3. At that point, light rope 2 at the other end.
4. In exactly fifteen minutes, rope 2 will be completely burnt.

Questions and Curiosity

- What did you enjoy?
- What was tricky?
- What would you like to build next?